Oral — AI Deployment : Renseignement OSINT par NER

Fiche de préparation. Durée cible : **6-8 min** de présentation + questions. Ne pas lire mot à mot : ce sont des points d'appui.

Plan chronométré

Temps	Section	Message clé
0:30	Accroche & problème	Transformer 21 000 articles bruts en renseignement exploitable
1:00	Cadre OSINT	Source TASS = biais assumé, on extrait « ce qu'une source affirme »
1:30	Pipeline	5 étapes : texte → annotation → NER → inférence → dashboards
1:30	Choix technique fort	**NER plutôt que LLM en prod** + transfer learning
1:00	Résultats	F1 0.973, top entités, tendance temporelle
1:00	Limites & honnêteté	MIL_UNIT faible, faux positifs, biais de source
0:30	Conclusion	Chaîne complète, reproductible, souveraine

Script parlé (points d'appui)

1. Accroche. « On part d'un corpus de ~21 000 articles de guerre de l'agence russe TASS, au format JSON. L'objectif : construire une chaîne complète qui va du texte brut à des dashboards de renseignement — repérer quelles armes, quelles unités et quelles organisations sont citées, et comment ça évolue dans le temps. »

2. Cadre OSINT (à ne pas zapper — c'est ce qui distingue un projet de renseignement). « Point méthodologique essentiel : TASS est un média d'État. On n'extrait jamais "la vérité", on extrait ce que la source affirme. La fiabilité et l'intention de la source font partie de l'analyse. »

3. Le pipeline en 5 étapes.

- Extraction : parsing JSON, nettoyage minimal, un texte propre par article.
- Annotation au format spaCy : `("texte", {"entities": [(start, end, LABEL)]})`. Pré-annotation par LLM, puis **validation humaine des offsets** (obligatoire).
- Fine-tuning d'un modèle NER sur 3 labels : **WEAPON, MIL_UNIT, MIL_ORG**.
- Inférence sur le corpus → entités regroupées par type.
- Ingestion Elasticsearch + dashboards Kibana.

4. Le choix technique à défendre : NER, pas LLM. « En production on veut du **déterminisme** (même entrée → même sortie, crucial pour la traçabilité du renseignement), de la **vitesse et un coût nul** (des milliers d'articles en local, sans API payante), et de la **souveraineté** (aucune donnée sensible envoyée à un tiers). Le LLM, lui, sert à pré-annoter le dataset d'entraînement : il crée les données, le NER fait le travail de production. » « On part de `en_core_web_sm`, le petit modèle anglais de spaCy : c'est du **transfer learning**, on réutilise un modèle déjà compétent sur les entités génériques plutôt que d'entraîner à zéro. »

5. Résultats. « Sur le jeu de dev (split 80/20), F1 global de **0.97**. Par type : WEAPON et MIL_ORG à 0.99, MIL_UNIT plus faible à 0.79. À l'inférence sur 3 000 articles : les armes dominantes sont drones, missiles, artillerie — une guerre de frappes à distance ; côté organisations, le *Defense Ministry* écrase tout, signature d'un corpus de communiqués officiels. La chronologie montre un pic mi-2024. »

6. Limites (les assumer = points bonus). « MIL_UNIT est plus dur : frontières ambiguës, vocabulaire variable — c'est la limite attendue du NER sur le contexte. J'ai un faux positif récurrent, "the company", confondu avec l'unité militaire. Et le biais de source reste présent dans toute interprétation. »

7. Conclusion. « Résultat : une chaîne complète texte → renseignement, reproductible en une commande (`make pipeline`), qui tourne en local. »

Chiffres à retenir (par cœur)

- Corpus : **21 704** articles (2015-2025), source **TASS**.
- Annotation : **400** articles, offsets validés. Split **80/20**.
- Schéma : **3 labels** — WEAPON, MIL_UNIT, MIL_ORG.
- Modèle de base : **en_core_web_sm** (transfer learning).
- **F1 = 0.973** ; WEAPON 0.988 · MIL_ORG 0.991 · MIL_UNIT 0.786.
- Inférence : **3 000** articles, **13 502** entités.
- Top armes : missile, drones, artillery, tanks, himars.

- Top orgs : Defense Ministry (~900), Navy, Russian Armed Forces, NATO.

Questions probables du jury + réponses

« Un F1 de 0.97, ce n'est pas trop beau ? » (LA question piège) → « C'est un score honnête mais à nuancer : mon jeu de dev suit la même distribution que l'entraînement, et une partie de l'annotation vient de règles. Le modèle apprend donc surtout à reproduire un schéma cohérent — le score mesure la **cohérence**, pas la généralisation absolue. Pour une évaluation stricte, il faudrait un jeu de test annoté à la main, en or, indépendant. »

« Pourquoi un NER et pas simplement un LLM ? » → Déterminisme (traçabilité), coût/vitesse à l'échelle, sortie structurée native, souveraineté. Le LLM reste utile pour pré-annoter. C'est un compromis assumé.

« Pourquoi seulement 3 labels ? » → Assez expressif pour du renseignement utile, assez simple pour être annoté et entraîné dans le temps d'un TP. La cohérence de l'annotation compte plus que l'exhaustivité.

« Comment gérez-vous le biais de TASS ? » → On ne le corrige pas, on le documente. La prédominance des communiqués du ministère est autant un fait qu'un marqueur de biais. Tout constat doit être recoupé avec des sources indépendantes.

« Comment avez-vous validé les annotations ? » → Recalcul et contrôle automatique des offsets sur le texte source (`step2_validate.py`), plus relecture manuelle d'un échantillon. spaCy refuse les entités mal alignées ou qui se chevauchent.

« MIL_UNIT est faible, comment l'améliorer ? » → Plus de données annotées, des contre-exemples (« the company » ≠ unité), règles d'annotation plus strictes (exiger un numéral/ordinal), ou un modèle plus gros. C'est le point d'amélioration prioritaire.

« C'est quoi le transfer learning ici concrètement ? » → Dans la config spaCy : `[components.ner] source = "en_core_web_sm"`. On initialise le composant NER avec les poids du modèle pré-entraîné au lieu d'un composant vierge, puis on ré-entraîne sur nos 3 labels.

« Pourquoi Elasticsearch et pas juste un script ? » → Recherche plein-texte rapide, **agrégations côté serveur** (top entités, séries temporelles, co-occurrences), et dashboards Kibana sans coder. Une pile de JSON ne permet pas l'analyse interactive.

CONFIDENTIAL

« 400 exemples, ce n'est pas trop peu ? » → Suffisant en transfer learning pour un schéma restreint et pour la démo ; le pipeline monte en volume sans changer de code (il suffit d'annoter plus).

« Le `.spacy` / DocBin, à quoi ça sert ? » → Format binaire de spaCy qui sérialise les Docs annotés ; c'est ce que consomme `spacy train`. On convertit les annotations JSON → DocBin après le split.

« Reproductibilité ? » → Seed fixe pour le split, `requirements.txt` figé, `make install` + `make pipeline`. Un collègue clone le repo et relance tout.

Pièges à éviter

- Ne pas survendre le 0.97 : le nuancer avant que le jury le fasse.
- Ne pas oublier le **cadre OSINT / biais de source** : c'est le cœur du sujet.
- Ne pas dire « le NER comprend le texte » : il **reconnaît des motifs statistiques**, il ne comprend pas.
- Assumer les limites (MIL_UNIT, faux positifs) : ça montre de la maturité.

Points forts à mettre en avant

- Chaîne **complète et reproductible** (une commande).
- Choix **NER vs LLM** argumenté (déterminisme, coût, souveraineté).
- **Validation des offsets** automatisée + humaine.
- Dashboards réels ingérés dans Elasticsearch, pas des maquettes.
- **Honnêteté méthodologique** (biais, limites, score à nuancer).

CONFIDENTIAL